

Android CI on Jenkins

User manual — setup, day-to-day usage, and troubleshooting.

1. What this system is

Jenkins builds Android projects inside disposable Docker containers, so builds don't depend on any developer's locally-installed JDK or Android SDK. There are two ways a project gets built:

- **A project with its own CI files** — some projects (e.g. an existing project you've already set up) carry their own Dockerfile and Jenkinsfile checked into that project's repo.
- **Any other Android project, onboarded with zero repo changes** — built through this shared setup: one generic Docker image plus a Jenkins Shared Library, both living in this `jenkins-ci-tooling` repo. Onboarding a project touches *zero files* in that project's own repository — the whole pipeline is configured inside Jenkins itself.

Both approaches produce a debug APK, run unit tests, and publish results the same way.

2. One-time setup on a new machine

Prerequisites: Docker installed and running; Jenkins installed and running as a service, with the Pipeline, Git, JUnit, Credentials Binding, and Docker Pipeline plugins (these ship with most default Jenkins installs — if Docker Pipeline isn't present, install it from Manage Jenkins → Plugins).

- 1 **Clone this repo** somewhere on the machine running Jenkins:

```
git clone <this-repo-location> /path/to/jenkins-ci-tooling
```

- 2 **Allow Jenkins to check out local git repositories.** This repo, and any per-project repos you point Jenkins at via a local path, live on local disk rather than a remote host — Git treats that as a security boundary by default:

```
sudo git config --system --add safe.directory '*'
```

- 3 **Allow Jenkins' Git plugin to check out from local filesystem paths.** Blocked by default as a separate security guard:

```
sudo mkdir -p /etc/systemd/system/jenkins.service.d
sudo tee /etc/systemd/system/jenkins.service.d/override.conf > /dev/null <<'EOF'
[Service]
Environment="JAVA_OPTS=-Djava.awt.headless=true -
Dhudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT=true"
EOF
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

If your Jenkins install already sets other `JAVA_OPTS`, merge them into this line instead of replacing it. If every project you'll build is hosted on GitHub/GitLab/etc. rather than a local path, steps 2 and 3 aren't strictly required — doing them anyway is harmless.

4 Build the generic Docker image once:

```
docker build -t android-generic-build:latest /path/to/jenkins-ci-tooling/docker/android-generic-image
```

Re-run this whenever the Dockerfile changes.

5 Register the Shared Library in Jenkins: Manage Jenkins → System → **Global Trusted Pipeline Libraries** → Add:

- Name: `android-ci`
- Default version: `main`
- Retrieval method: Modern SCM → Git
- Project Repository: `/path/to/jenkins-ci-tooling` (wherever you cloned it)
- Credentials: none needed for a local path
- Load implicitly: leave unchecked

Save.

You're now ready to build any Android project.

3. Onboarding a new Android project into Jenkins

Entirely done in the Jenkins UI. Nothing gets added to the new project's own repository.

Step 1 — Credential

Decide whether to reuse an existing Jenkins credential or create a new one, scoped to the new repo:

- **Manage Jenkins** → **Credentials** → Add Credentials
- Kind: *Username with password*
- Username: your Git host username
- Password: a fine-grained Personal Access Token, scoped to only that repo, with **read-only content** permission

Step 2 — Create the Jenkins job

- 1 **New Item** → give it a name → type **Pipeline** → OK.
- 2 Under **Pipeline**, set Definition to **Pipeline script** (inline — *not* "Pipeline script from SCM").
- 3 Paste the script below, filling in that project's repo URL and the credential ID from Step 1.
- 4 Save, then **Build Now**.

```
@Library('android-ci') _

androidBuildPipeline(
    repoUrl: 'https://github.com/<org>/<new-project>.git',
    credentialsId: 'the-credential-id-you-picked'
)
```

That's the entire job. It checks out the repo, runs `assembleDebug` and `testDebugUnitTest`, publishes JUnit results, and archives a renamed APK — without any Dockerfile or Jenkinsfile existing in that project's repo.

Overriding the defaults

If the new project doesn't follow the standard single-`app`-module convention, add any of these named parameters to the same call:

Parameter	Default	When to override
<code>branch</code>	<code>main</code>	Project uses a different default branch
<code>buildTask</code>	<code>assembleDebug</code>	Different build variant/flavour
<code>testTask</code>	<code>testDebugUnitTest</code>	Different test variant
<code>apkGlob</code>	<code>app/build/outputs/apk/debug/*.apk</code>	Module isn't named <code>app</code> , or different variant path
<code>projectName</code>	derived from the repo URL	Want a different name in the archived APK filename
<code>image</code>	<code>android-generic-build:latest</code>	Project needs a different JDK/base image

Example — a project called `widget-scanner` that builds a `release` flavour out of a `widget-app` module on a `develop` branch, and needs a newer JDK image:

```
@Library('android-ci') _

androidBuildPipeline(
    repoUrl: 'https://github.com/someorg/widget-scanner.git',
    credentialsId: 'widget-scanner-readonly',
    branch: 'develop',
    buildTask: 'assembleRelease',
    testTask: 'testReleaseUnitTest',
    apkGlob: 'widget-app/build/outputs/apk/release/*.apk',
    projectName: 'widget-scanner',
    image: 'android-generic-build:jdk21'
)
```

Every parameter here is optional and independent — override only the ones that differ from the defaults in the table above, and drop the rest. `repoUrl` and `credentialsId` are the only two that are always required.

4. Building locally, without Jenkins (Docker only)

Two options, depending on the project:

A project with its own build script

Some projects (e.g. one that already has its own Dockerfile/Jenkinsfile checked in) carry their own convenience script, typically `ci/build-apk.sh` in that project's repo. Run it from inside that project's checkout:

```
./ci/build-apk.sh
```

Any Android project — the generic script

Works for literally any Android project, with no per-project script needed. Lives in this `jenkins-ci-tooling` repo:

```
./scripts/build-apk.sh <git-repo-url-or-local-path> [gradle-task]

# Examples:
./scripts/build-apk.sh https://github.com/someorg/some-app.git
./scripts/build-apk.sh ~/code/some-app assembleRelease
```

Clones the repo if given a URL (into a temporary directory, cleaned up afterward — copy the APK out before it exits if you need it kept), or builds an existing local checkout in place if given a path. Prints the path to the resulting APK when done.

Both approaches require only Docker installed — no JDK, no Android SDK — and share the same cache volumes as the Jenkins jobs, so a warm cache benefits every build regardless of which entry point you use.

5. APK naming (Jenkins-built projects)

Every archived APK from a project onboarded through the shared library is automatically renamed to:

```
<project-name>-<version-name>-<build-date>-<variant>.apk
```

Example from a real build: `mobility-tracker-poc-0.1-m1-20260831-debug.apk`

Project name comes from the repo URL (or the `projectName` override), version comes from the app's own `versionName` (read automatically from AGP's build metadata — no project-specific configuration needed), date is the build date, and variant reflects the build type/flavour actually built.

6. Why builds get faster after the first run

Two things are cached in named Docker volumes shared across every project built this way:

- **Android SDK components** — platforms and build-tools are downloaded on first use and reused by every subsequent build, of any project, that needs the same version.

- **Gradle's dependency cache** — the Gradle distribution itself and common libraries (Kotlin stdlib, AndroidX, etc.) are downloaded once and reused.

Because the cache is shared across projects, two different projects' builds running at the exact same moment can briefly wait on each other (a lock, not a failure or corruption) — this is expected and was verified deliberately. It resolves itself within seconds to a couple of minutes depending on what's being downloaded.

7. Troubleshooting

Symptom	Cause & fix
detected dubious ownership in repository	Git's ownership-mismatch protection. Covered by setup step 2 (<code>git config --system --add safe.directory '*'</code>). If it reappears, that command may not have actually run — re-run it and confirm no error.
Checkout ... aborted because it references a local directory	Jenkins' Git plugin blocks local-path checkouts by default. Covered by setup step 3. Confirm the running Jenkins process actually has the flag (check its command line) — a config file edit alone doesn't apply until Jenkins is restarted.
Unable to find Jenkinsfile from git	Only applies to a project using its own Dockerfile/Jenkinsfile (not the shared library path). Means changes were made locally but never committed and pushed to that project's remote.
SDK directory is not writable / a build-tools install fails	Should not happen — the generic image makes the SDK directory world-writable as its last build step. If it does, the image may need rebuilding (step 4 of setup).
403 / "Write access to repository not granted" on checkout	The credential's token doesn't have read permission on that specific repo, or the token needs regenerating after a permission change.
Build seems to hang partway through	Likely waiting on the shared cache lock because another project's build is running at the same time — see Section 6. Check the Jenkins dashboard for other active builds.
Missing required parameter for agent type "docker": image	A shared-library config using a local variable literally named <code>image</code> / <code>args</code> /etc. collides with the declarative pipeline's own directive keywords. Rename the variable if you're editing <code>vars/androidBuildPipeline.groovy</code> .

Appendix — file & resource locations

What	Location
Generic build image	<jenkins-ci-tooling>/docker/android-generic-image/Dockerfile
Shared pipeline library step	<jenkins-ci-tooling>/vars/androidBuildPipeline.groovy
Generic QA build script	<jenkins-ci-tooling>/scripts/build-apk.sh
Rebuild the generic image	<code>docker build -t android-generic-build:latest <jenkins-ci-tooling>/docker/android-generic-image</code>
Shared SDK cache volume	Docker volume: <code>android-sdk-cache</code>

Shared Gradle cache volume	Docker volume: android-gradle-cache
Global Trusted Pipeline Library registration	Manage Jenkins → System → Global Trusted Pipeline Libraries → name "android-ci"
Local-checkout security override	/etc/systemd/system/jenkins.service.d/override.conf
A project with its own CI files (example)	that project's own repo root: Dockerfile under <code>ci/</code> , Jenkinsfile at repo root, QA script at <code>ci/build-apk.sh</code>